- Asynchronous code—From the top, the `async` keyword allows this function to call asynchronous code. We need the `await` keyword when calling `data(from:)` to break out of the current thread and return asynchronously. This differs from Ruby, where most code is *blocking* - calling Ruby code that makes an HTTP request will "wait" until it returns. In Swift, we have to be a bit more explicit.

- Error handling—`do`, `try`, and `catch` are Swift's approach to error handling. Functions that can raise errors must be called with the `try` keyword. And anything that calls these functions must be wrapped in a `do`/`catch` block. `catch` handles any errors that arise, automatically exposing the `error` that occurred. These map pretty closely to `begin` and `rescue` in Ruby.

- Ignored variables—`data(from:)` returns a tuple with a reference to the data and HTTP response. We only care about the data, so we can use an underscore to ignore the other element, just like in Ruby.

Parse the JSON and Create a Hike

Use a `JSONDecoder` to decode the JSON response directly into a `Hike` instance. This call can also raise an error, so don't forget the `try` keyword:

ch05_17/ios/App/ViewModels/HikeViewModel.swift

```
import Foundation

@Observable class HikeViewModel {
    var hike: Hike?

    private let url: URL

    init(url: URL) {
        self.url = url.appendingPathExtension("json")
    }

    func fetchCoordinates() async {
```